

Tasca Sprint 4

1. Nivell 1: Entorn i Ingesta Híbrida (Code-First)

Exercici 1: Consulta sobre Taula no Optimitzada (Diagnòstic)

El Country Manager d'Alemanya necessita revisar urgentment les transaccions del dia 12 de març de 2022.

✚ Tasca:

- » 1. Escribe la consulta que uneix (JOIN) transaccions i companyies.
- » 2. Filtra els resultats per la data indicada i el país "Germany".
- » 3. Sense executar la consulta, realitza un "Dry Run" (auditoria de costos).

👁️ **Observació:** Fixa't que BigQuery llegeix gairebé tota la taula tot i demanar només un dia (Full Table Scan).

```
39
40 SELECT tc.transaction_id,
41         c.company_name,
42         c.country
43 FROM `sprint3_silver.transactions_clean` AS tc
44 JOIN `sprint3_silver.companies_clean` AS c
45 ON tc.business_id = c.company_id
46 WHERE declined = 0;
47
48 --
```

✅ Esta consulta procesará 5.15 MB cuando se ejecute.

Imagen 1

```
39 SELECT tc.transaction_id,
40         c.company_name,
41         c.country
42 FROM `sprint3_silver.transactions_clean` AS tc
43 JOIN `sprint3_silver.companies_clean` AS c
44 ON tc.business_id = c.company_id
45 WHERE c.country = 'Germany' AND
46        date(`timestamp`) = '2022-03-12'
47 AND declined = 0;
48
```

✅ Esta consulta procesará 5.92 MB cuando se ejecute.

Imagen 2

En la primera imagen (imagen 1) se muestra el coste que tendríamos en ejecutar la query simplemente con la join entre las dos tablas, mientras que en la segunda imagen (imagen 2) observamos el coste que tendríamos al ejecutar la query con, además de la join, filtrando por fecha y país. Observamos que los bytes procesados en ambas queries son parecidos, siendo ligeramente mayor en la segunda. Consecuentemente, esto implicaría un coste parecido al ejecutar cualquiera de las dos queries, independientemente se usen o nos filtros.

Exercici 2: Re-arquitectura i Optimització de l'Emmagatzematge (Partition & Cluster)

Tasca:

Pas 1: Generació de Dades Recents (Mocking Data)

Crea una taula intermèdia anomenada `sprint3_silver.transactions_recent` a partir de la taula `sprint3_silver.transactions_clean`. El teu objectiu és mantenir totes les columnes, però substituir el timestamp original per un de nou, generat aleatòriament perquè caigui dins dels últims 50 dies.

» **Pista Tècnica:** Per fer-ho sense escriure totes les columnes a mà, pots utilitzar `SELECT * EXCEPT(timestamp)`. Per calcular la data aleatòria, hauràs de restar una quantitat de dies a l'instant actual. Et seran molt útils les funcions `TIMESTAMP_SUB()`, `CURRENT_TIMESTAMP()`, i la combinació `CAST(RAND() * 50 AS INT64)` per generar els dies de desfasament.

Pas 2: Creació de la Taula Optimitzada (Partitioning & Clustering)

Ara, crea la teva taula definitiva `sprint3_gold.fact_transactions_optimized` a partir de les dades recents que acabes de generar a `transactions_recent`. Has de construir la sentència DDL per configurar la taula amb les següents estratègies físiques d'emmagatzematge:

» **Particionament (Partitioning):** Divideix la taula per la data del camp `DATE(timestamp)`. Això permetrà que les consultes que filtrin per dia (`WHERE date = ...`) només llegeixin la partició necessària.

» **Clustering:** Ordena les dades dins de cada partició per `business_id`. Això accelerarà dràsticament els filtres per companyia i els encreuaments (JOINS) amb la dimensió d'empreses.

Entrega:

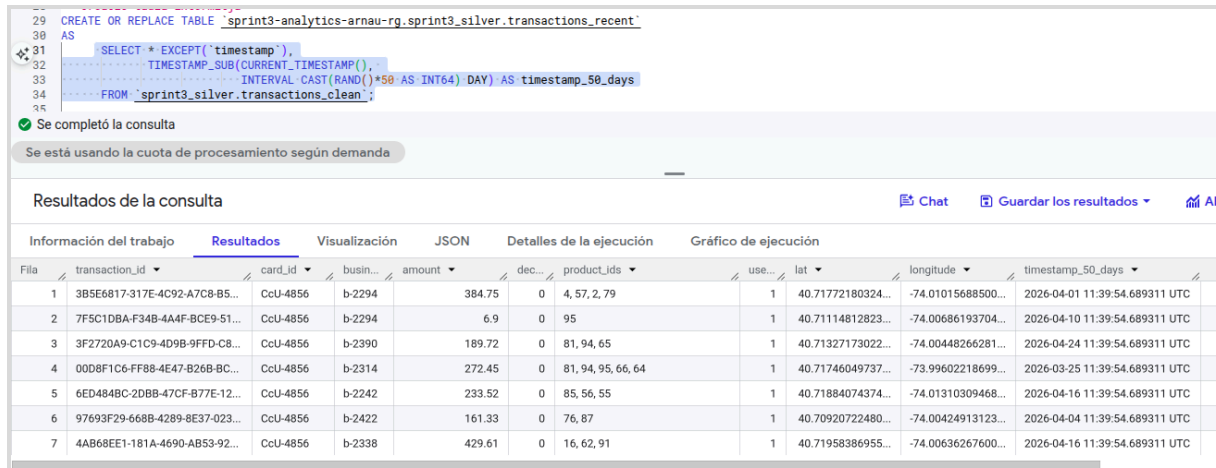
- » El codi SQL utilitzat per al Pas 1.
- » El codi DDL (`CREATE OR REPLACE TABLE...`) utilitzat al Pas 2.
- » Una captura de pantalla dels "Detalls" de la nova taula `fact_transactions_optimized` on es pugui comprovar que està particionada i agrupada en clústers correctament.

La primera imagen (imagen 3) muestra el contenido que tendrá la nueva tabla. Hemos utilizado `EXCEPT` para indicar que queremos todas las columnas excepto ``timestamp``. Para crear la nueva columna tipo ``timestamp`` que la hemos llamado ``timestamp_50_days``, hemos seguido la siguiente lógica:

- `TIMESTAMP_SUB`: esta función resta una cantidad específica de tiempo a un dato tipo `TIMESTAMP` y recibe dos parámetros; la expresión original de la cual restamos y la cantidad de tiempo a restar en una unidad de tiempo.
- En este caso, hemos utilizado `CURRENT_TIMESTAMP()` como la expresión original sobre la cual hemos de sustraer (tiempo actual).
- En el segundo parámetro, le hemos indicado que nos reste un número aleatorio entero (entre 0 y 50) respecto a la fecha actual, es decir, que para cada registro

obtendremos una fecha entre la fecha actual y 50 días atrás. La resta representa los días, pues esta unidad de tiempo le hemos indicado.

- Para ello hemos usado la función RAND(), que nos proporciona valores aleatorios entre 0 y 1, y hemos multiplicado este valor por 50, pues es el tiempo máximo de días respecto al cual queremos obtener fechas.
- Finalmente, casteamos a INT64 ya que INTERVAL no acepta valores decimales, que es el tipo de dato que obtuvimos al usar RAND().

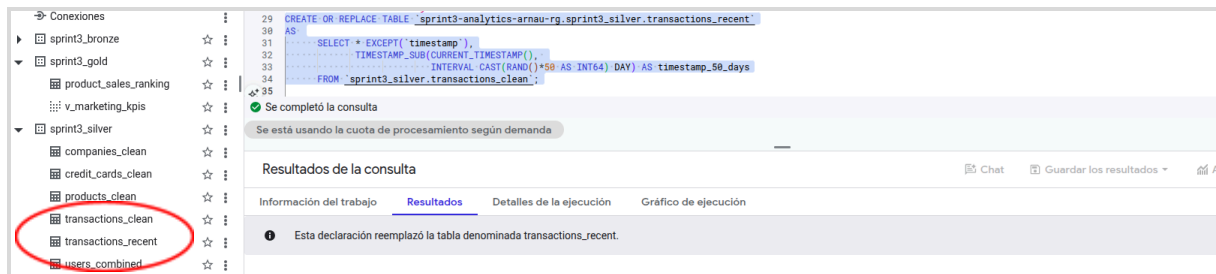


The screenshot shows a BigQuery console interface. At the top, a SQL query is displayed, creating a new table 'transactions_recent' by selecting data from 'transactions_clean' and adding a new timestamp column 'timestamp_50_days' calculated using a random value between 0 and 50 days in the past. Below the query, a status message indicates the query completed successfully. The main part of the screenshot shows the 'Results' tab with a table of 7 rows. The columns include transaction_id, card_id, business_id, amount, decimal, product_ids, user_id, latitude, longitude, and timestamp_50_days.

Fila	transaction_id	card_id	business_id	amount	dec...	product_ids	use...	lat	longitude	timestamp_50_days
1	3B5E6817-317E-4C92-A7C8-B5...	CcU-4856	b-2294	384.75	0	4, 57, 2, 79	1	40.71772180324...	-74.01015688500...	2026-04-01 11:39:54.689311 UTC
2	7F5C1DBA-F34B-4A4F-BCE9-51...	CcU-4856	b-2294	6.9	0	95	1	40.71114812823...	-74.00686193704...	2026-04-10 11:39:54.689311 UTC
3	3F2720A9-C1C9-4D9B-9FFD-C8...	CcU-4856	b-2390	189.72	0	81, 94, 65	1	40.71327173022...	-74.00448266281...	2026-04-24 11:39:54.689311 UTC
4	00D8F1C6-FF88-4E47-B26B-BC...	CcU-4856	b-2314	272.45	0	81, 94, 95, 66, 64	1	40.71746049737...	-73.99602218699...	2026-03-25 11:39:54.689311 UTC
5	6ED484BC-2DBB-47CF-B77E-12...	CcU-4856	b-2242	233.52	0	85, 56, 55	1	40.71884074374...	-74.01310309468...	2026-04-16 11:39:54.689311 UTC
6	97693F29-668B-4289-8E37-023...	CcU-4856	b-2422	161.33	0	76, 87	1	40.70920722480...	-74.00424913123...	2026-04-04 11:39:54.689311 UTC
7	4AB68EE1-181A-4690-AB53-92...	CcU-4856	b-2338	429.61	0	16, 62, 91	1	40.71958386955...	-74.00636267600...	2026-04-16 11:39:54.689311 UTC

Imagen 3

En la siguiente imagen, mostramos la ejecución de la query que da lugar a la creación de la nueva tabla. Hemos usado la sentencia CREATE OR REPLACE TABLE. En la izquierda observamos la nueva tabla dentro de la capa Silver.



The screenshot shows the BigQuery console with the same SQL query as in Image 3. On the left, the 'Conexiones' (Connections) pane shows a tree view of the database schema. The 'sprint3_silver' folder is expanded, and the 'transactions_recent' table is highlighted with a red circle. The main panel shows the query results, including a message: 'Esta declaración reemplazó la tabla denominada transactions_recent.' (This statement replaced the table named transactions_recent.)

Imagen 4

En la imagen que sigue (Imagen 5) vemos la query que crea la nueva tabla optimizada. Esta tabla, gracias a la partición y a la agrupación en clústeres que hemos realizado, aumenta la eficiencia y, consecuentemente, reduce el coste, a la hora de hacer cualquier búsqueda o consulta. Para la partición, hemos usado los días de la columna 'timestamp_50_dias', mientras que la agrupación la hemos hecho por el campo 'business_id'. Hemos decidido añadir todos los campos a la nueva tabla para tener más a mano todos los datos y no tener que hacer ninguna join.



Imagen 5

En la Imagen 6 observamos los detalles de la tabla recién creada. Vemos, por ejemplo, que se encuentra en la capa Gold y que la ubicación de la tabla es en EU (European Union). Más abajo, podemos verificar los cambios realizados: que está particionada por días en función del campo 'timestamp_50_days' y que está agrupada por el campo 'business_id'.

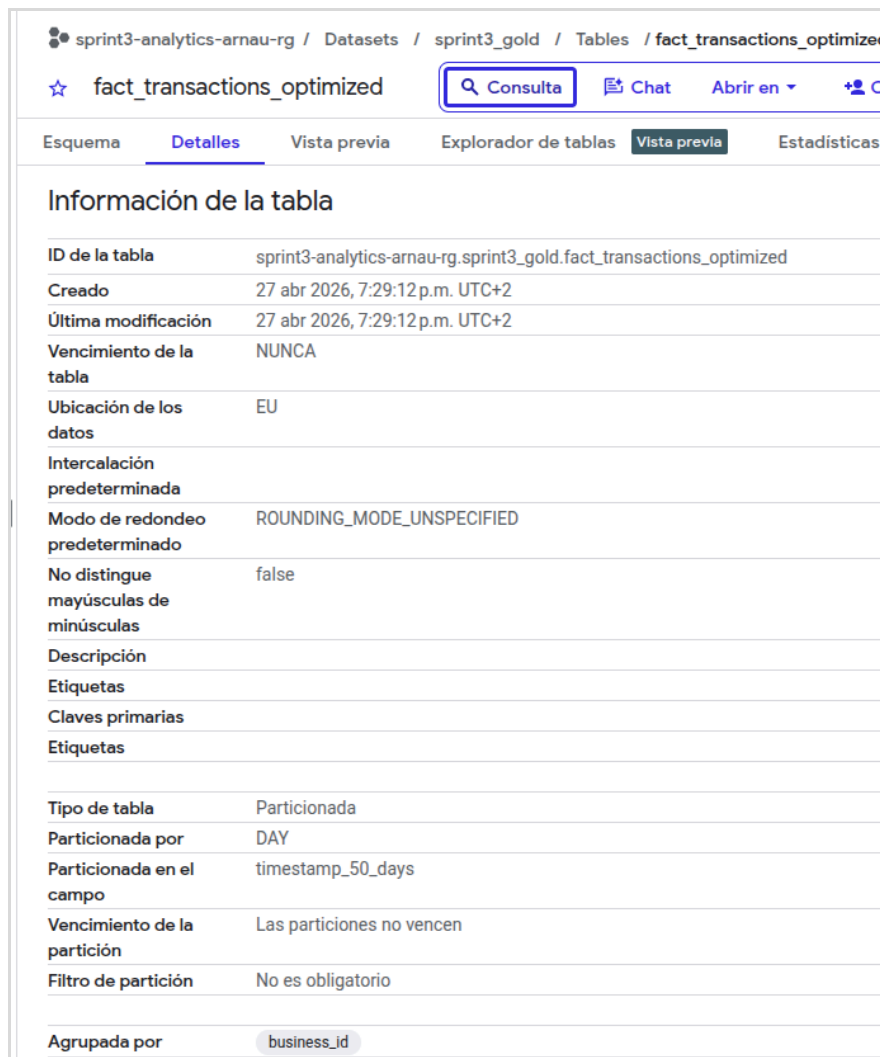


Imagen 6

Exercici 3: La Prova del Cotó (Benchmark)

✂ **Tasca:** L'objectiu és clar i directe: aplicar exactament la mateixa consulta a dues taules diferents i comparar-ne el cost computacional.

Construeix una consulta SQL que seleccioni totes les columnes (SELECT *) i filtri les dades dels últims 30 dies.

⚠ NO executis les consultes encara. En aquest exercici només utilitzarem el validador de BigQuery ("Dry Run", a dalt a la dreta de l'editor) per visualitzar els Bytes processats de manera gratuïta.

Fes la comparativa seguint aquests dos passos:

» **Pas 1 (Taula no optimitzada):** Apunta la teva consulta a la taula `sprint3_silver.transactions_recent`. Observa el validador i fes una captura de pantalla dels bytes que processaria.

» **Pas 2 (Taula optimitzada):** Sense canviar res de la lògica de la teva consulta, modifica només el FROM perquè ara apunti a la taula particionada `sprint3_gold.fact_transactions_optimized`. Observa com canvia el validador i fes la segona captura.

👉 Entrega:

- » El codi SQL base que has dissenyat.
- » Les dues captures de pantalla on es vegi la diferència de "Bytes processats" al validador.
- » Un breu text calculant quin és el % d'estalvi que has aconseguit (compara els bytes del Pas 1 respecte als del Pas 2).

En la primera imagen (imagen 7) mostramos los bytes que se procesarían al ejecutar la query en la tabla 'transactions_recent'; mientras que en la segunda imagen (imagen 8), se observan los que se procesarían en la tabla particionada y agrupada por clusters, la tabla 'fact_transactions_optimized'.

La consulta en la tabla no optimizada consumiría 10,87MB, mientras que la consulta en la tabla optimizada consumiría 4,69MB. Si 10,87 representa el 100% de consumo, i 4,69 el consumo con el que comparamos, para saber el porcentaje de ahorro, hemos de calcular la diferencia y hacer una regla de 3. Así, podemos afirmar que nos ahorraríamos un 56,85% de MB procesados.

```

54 SELECT *
55 FROM `sprint3-analytics-arnau-rg.sprint3_silver.transactions_recent`
56 WHERE DATE(timestamp_50_days) < TIMESTAMP_SUB(DATE(CURRENT_TIMESTAMP()), INTERVAL 30 DAY);
57
58 SELECT *
59 FROM `sprint3-analytics-arnau-rg.sprint3_gold.fact_transactions_optimized`
60 WHERE DATE(timestamp_50_days) < TIMESTAMP_SUB(DATE(CURRENT_TIMESTAMP()), INTERVAL 30 DAY);
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80

```

✓ Esta consulta procesará 10.87 MB cuando se ejecute.

Imagen 7

```

58 SELECT *
59 FROM `sprint3-analytics-arnau-rg.sprint3_gold.fact_transactions_optimized`
60 WHERE DATE(timestamp_50_days) < TIMESTAMP_SUB(DATE(CURRENT_TIMESTAMP()), INTERVAL 30 DAY);
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80

```

✓ Esta consulta procesará 4.69 MB cuando se ejecute.

Imagen 8

Exercici 4: Smart Caching (Vistes Materialitzades)

🧩 **Tasca:** Crea una Vista Materialitzada anomenada `sprint3_gold.mv_daily_sales` que mostri les Vendes Totals per Dia.

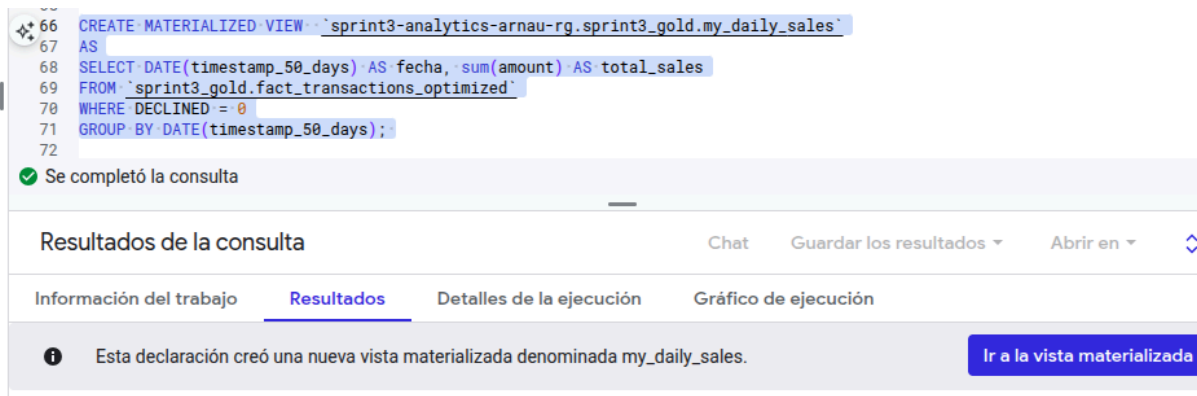
👉 Entrega:

- » Codi per Crear la Vista Materialitzada.
- » Fes una consulta que mostri la vista creada i fes una captura de pantalla que mostri en el seu conjunt la vista materialitzada creada, amb la consulta executada i els bits processats.

En la imatge 9 se observa el codi utilitzat per crear la vista materialitzada. Hem agrupat per data i utilitzat la funció d'agregació `SUM(amount)` per obtenir el total de vendes per dia.

A continuació, en la imatge 10, observem una consulta en la mateixa taula, on podem diferenciar en el panel de l'esquerra, la vista creada i situada dins de la capa Gold. També observem el contingut de la vista materialitzada: la taula amb el nombre de vendes per dia.

Finalment, en la imatge 11, observem els bytes processats al executar la consulta.



Imatge 9

The screenshot shows the Google Cloud BigQuery console. On the left, a project tree is visible with folders for 'sprint3_bronze', 'sprint3_gold', and 'sprint3_silver'. The 'sprint3_gold' folder is expanded, showing tables like 'dim_transactions_flat', 'fact_transactions_optimized', 'my_daily_sales', 'product_sales_ranking', 'v_marketing_kpis', 'companies_clean', 'credit_cards_clean', 'products_clean', 'transactions_clean', 'transactions_recent', and 'users_combined'. The 'my_daily_sales' table is selected. In the center, a SQL query is shown: `SELECT * FROM `sprint3-analytics-arnau-rg.sprint3_gold.my_daily_sales`;`. Below the query, a green checkmark indicates 'Se completó la consulta' (Query completed). A message states 'Se está usando la cuota de procesamiento según demanda' (Using the on-demand processing quota). The 'Resultados de la consulta' (Query results) section is active, displaying a table with 9 rows and 3 columns: 'Fila' (Row), 'fecha' (date), and 'total_sales'. The results show data for dates from 2026-03-08 to 2026-03-16. At the bottom right, it says 'Resultados por página' (Results per page).

Fila	fecha	total_sales
1	2026-03-08	279976.41
2	2026-03-09	514111.8300000...
3	2026-03-10	514089.0100000...
4	2026-03-11	532929.1100000...
5	2026-03-12	511548.5099999...
6	2026-03-13	525019.0400000...
7	2026-03-14	519368.2500000...
8	2026-03-15	516569.5400000...
9	2026-03-16	517648.7700000...

Imagen 10

The screenshot shows the 'Información del trabajo' (Job information) tab in the BigQuery console. It displays details for a completed query job. The job ID is 'sprint3-analytics-arnau-rg:EU.bqjob_1f9f9628_19dea3ac61'. The user is 'rgnadata@gmail.com'. The job was created on May 2, 2026, at 9:46:53 p.m. UTC+2. It started and finished at 9:46:54 p.m. UTC+2. The duration was 0 seconds. The job processed 816 B of data and billed 10 MB. The job completed 26 milliseconds before the next scheduled run.

Información del trabajo	Resultados	Visualización	JSON	Deta
ID de trabajo	sprint3-analytics-arnau-rg:EU.bqjob_1f9f9628_19dea3ac61			
Usuario	rgnadata@gmail.com			
Ubicación	EU			
Hora de creación	2 may 2026, 9:46:53 p.m. UTC+2			
Hora de inicio	2 may 2026, 9:46:54 p.m. UTC+2			
Hora de finalización	2 may 2026, 9:46:54 p.m. UTC+2			
Duración	0 s			
Bytes procesados	816 B			
Bytes facturados	10 MB			
Milisegundos de	26			

Imagen 11

2. Nivell 2

Exercici 1: Perfilat de Clients VIP (Mètriques Agregades amb CTEs)

Escenari: L'equip de Màrqueting vol analitzar el comportament dels nostres millors clients per dissenyar l'estratègia del pròxim any. Definim "VIP" com aquells amb una despesa acumulada superior a 500€. Necessiten un informe que, per a cada VIP, mostri el seu nom, contacte i el seu patró de compra: quantes vegades ha comprat, quant gasta de mitjana i quina va ser la seva compra rècord.

✂ Tasca:

- » 1. Crea una CTE anomenada VIP_Stats que agrupi per usuari i calculi:
 - La Despesa Total (SUM).
 - La Quantitat de Transaccions (COUNT).
 - El Tiquet Mitjà (AVG), arrodonit a 2 decimals.
 - La Compra Màxima (MAX).
 - Filtre: Mantén només aquells la Despesa Total dels quals sigui > 500.
- » Creua la CTE amb users_combined per obtenir les dades personals.

Requisits de Sortida:

- Columnes: user_id, nom_complet, email, num_compres, tiquet_mig, max_compra, total_gastat.
- Ordenat per total_gastat descendent.

```

33 WITH VIP_Stats AS (
34     SELECT user_id,
35            ROUND(SUM(amount),2) AS despesa_total,
36            COUNT(*) AS num_compras,
37            ROUND(AVG(amount),2) AS tiquet_mig,
38            MAX(amount) AS compra_maxima
39 FROM `sprint3-analytics-arnau-rg.sprint3_gold.fact_transactions_optimized`
40 WHERE declined = 0
41 GROUP BY user_id
42 HAVING despesa_total > 500)
43
44 SELECT d.user_id,
45        CONCAT(d.name, ' ', d.surname) AS nom_complet,
46        d.email,
47        vs.num_compras,
48        vs.tiquet_mig,
49        vs.compra_maxima,
50        vs.despesa_total
51 FROM `sprint3-analytics-arnau-rg.sprint3_silver.users_combined` AS d
52 JOIN VIP_Stats AS vs
53 ON d.user_id = vs.user_id
54 ORDER BY despesa_total DESC;
55

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#)

Fila	user_id	nom_complet	email	num_compras	tiquet_mig	compra_maxima	despesa_total
1	289	Dxwgi Hwcru	dxwgi.hwcru@example.com	91	446.91	554.71	40669.16
2	318	Bnyr Astuw	bnyr.astuw@example.com	86	449.0	615.51	38614.39
3	455	Fwfl Phylgnmhnd	fwfl.phylgnmhnd@example.com	66	522.6	724.44	34491.27
4	417	Fqatil Rdcjcvoa	fqatil.rdcjcvoa@example.com	73	421.87	675.78	30796.3
5	185	Molly Gilliam	donec@outlook.couk	107	274.72	659.31	29394.79
6	285	Brcxprnj Lnpna	brcxprnj.lnpna@example.com	70	417.85	630.52	29249.51

Imagen 12

Exercici 2: Anàlisi de Tendències (Window Functions sobre Vistes)

Escenari: La Direcció Financera vol monitoritzar la "velocitat de vendes" diària. Necessiten un informe que compari el rendiment de cada dia contra el dia anterior per detectar caigudes brusques o pics de creixement (Day-over-Day Growth).



Estratègia d'Optimització:

Per a aquest càlcul NO has d'anar a la taula de fets original ni recalculer sumes. Com a bon Data Engineer, has d'utilitzar la Vista Materialitzada `sprint3_gold.mv_daily_sales` que vas crear a l'exercici anterior. Això garanteix que l'anàlisi de tendències sigui instantània i no consumeixi recursos de còmput innecessaris.



Tasca:

Crea una consulta sobre la vista materialitzada que generi un informe amb les següents 4 columnes:

- » 1.Data: La data de venda.
- » 2.Vendes_Avui: El total venut aquell dia.
- » 3.Vendes_Ahir: El total venut el dia anterior (usant funcions de finestra).
- » 4.Diff_Percentual: El percentatge de creixement o decreixement respecte al dia anterior, arrodonit a 2 decimals.



Pista Tècnica:

Utilitza la funció `LAG(camp) OVER (ORDER BY data)` per llegir el valor de la fila anterior sense necessitat de fer JOINS complexos.

En este ejercicio hemos utilizado la función `LAG() OVER()` para generar una nueva columna llamada 'vendes_ahir' la cual nos sirve para comparar la venta del día con la venta del día anterior. Dentro de `LAG` hemos especificado el campo 'total_sales' que es el campo que queremos recuperar y en el `ORDER BY` le hemos colocado el campo 'fecha', el cual establece el orden a recuperar.

Para calcular el porcentaje de la diferencia entre la venta de un día y el día anterior, hemos agrupado todo lo anterior en un paréntesis y hemos hecho un `SELECT` sobre ello. En el `SELECT`, le hemos indicado que queremos todos los campos y también la diferencia en porcentaje, calculada como el $((\text{valor final} - \text{valor inicial}) / \text{valor inicial} * 100)$. Finalmente, hemos ordenado por fecha de manera ascendente, ya que el `ORDER BY` de la cláusula del `OVER` trabaja a nivel interno de la función, no a nivel externo.

La query final se puede observar en la imagen siguiente (Imagen 13).

```

113
114 SELECT *, ROUND(((vendes_avui - vendes_ahir) / vendes_ahir), 2) * 100 AS diff_percentual
115 FROM
116 (SELECT fecha,
117      ROUND(total_sales, 2) AS vendes_avui,
118      ROUND(LAG(total_sales) OVER (ORDER BY fecha ASC), 2) AS vendes_ahir
119 FROM `sprint3-analytics-arnau-rg.sprint3_gold.my_daily_sales`) AS t_vendas
120 ORDER BY fecha ASC;
121
122

```

✓ Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#) [Guardar los resultados](#)

Información del trabajo		Resultados	Visualización	JSON	Detalles de la ejecución	
Fila	fecha	vendes_avui	vendes_ahir	diff_percentual		
1	2026-03-08	279976.41	null	null		
2	2026-03-09	514111.83	279976.41	84.0		
3	2026-03-10	514089.01	514111.83	0.0		
4	2026-03-11	532929.11	514089.01	4.0		
5	2026-03-12	511548.51	532929.11	-4.0		
6	2026-03-13	525019.04	511548.51	3.0		
7	2026-03-14	519368.25	525019.04	-1.0		
8	2026-03-15	516569.54	519368.25	-1.0		
9	2026-03-16	517648.77	516569.54	0.0		
10	2026-03-17	497210.02	517648.77	-4.0		

Imagen 13

Exercici 3: Totals Acumulats (Running Totals sobre Vistes)

Escenari:

El Director Financer (CFO) necessita visualitzar la corba de creixement anual. No li interessa tant el que es va vendre avui, sinó quant portem facturat en total des de l'inici de l'any fins avui (YTD - Year To Date).

🌱 **Tasca:** Utilitzant la vista materialitzada mv_daily_sales (per no recalculer agregacions base), genera un informe amb tres columnes:

- » 1.Data.
- » 2.Vendes del Dia: Arrodonit a 2 decimals.
- » 3.Vendes Acumulades YTD: Una suma progressiva de les vendes que ha de reiniciar-se cada 1 de Gener. El resultat final ha d'estar arrodonit a 2 decimals.

💡 Pista Tècnica:

Per aconseguir el reinici anual, necessites dividir la finestra usant PARTITION BY EXTRACT(YEAR FROM data). L'estructura completa seria: SUM(...) OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW).

Para calcular las ventas acumuladas, hemos utilizado la función SUM() con 'total_sales' como parámetro. En el OVER, hemos utilizado PARTITION BY + EXTRACT(YEAR) para indicar que deseamos particionar por año, de manera que la suma acumulativa sea anual. En el ORDER BY hemos indicado que el orden a la hora de sumar sea ascendente y hemos enmarcado la función de ventana definiendo un rango de filas de manera que cada fila sólo debe sumar la fila actual más las filas precedentes, dentro de un mismo año (ver imagen 14).

```

124 SELECT fecha,
125 ROUND(total_sales) AS vendes_del_dia,
126 ROUND(SUM(total_sales) OVER (PARTITION BY EXTRACT(YEAR FROM fecha) ORDER BY fecha ASC, fecha ROWS BETWEEN UNBOUNDED
127 PRECEDING AND CURRENT ROW),2) AS vendes_acumulades
128 FROM `sprint3-analytics-arnau-rg.sprint3_gold.my_daily_sales`
129 ORDER BY fecha ASC;
  
```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat Guardar los resultados Abrir en

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	fecha	vendes_del_dia	vendes_acumula...		
1	2026-03-08	279976.0	279976.41		
2	2026-03-09	514112.0	794088.24		
3	2026-03-10	514089.0	1308177.25		
4	2026-03-11	532929.0	1841106.36		
5	2026-03-12	511549.0	2352654.87		
6	2026-03-13	525019.0	2877673.91		
7	2026-03-14	519368.0	3397042.16		
8	2026-03-15	516570.0	3913611.7		
9	2026-03-16	517649.0	4431260.47		
10	2026-03-17	497210.0	4928470.49		
11	2026-03-18	532674.0	5461144.44		
12	2026-03-19	509235.0	5970379.7		
13	2026-03-20	513670.0	6484049.38		

Resultados por página: 50 1 - 50 de 51

Imagen 14

Exercici 4: Fidelització i Valor del Client (Filtratge Avançat)

Escenari: L'equip de Màrqueting llança la campanya "A la tercera va la vençuda". Volen enviar un regal als usuaris que hagin completat la seva tercera transacció. No obstant això, el pressupost del regal no és fix: depèn de la "qualitat" del client en els seus inicis. Necessiten saber el Tiquet Mitjà d'aquestes 3 primeres compres per segmentar si envien un detall bàsic o un regal premium.

Tasca:

Genera un llistat dels usuaris que han arribat (o superat) la seva tercera compra, mostrant:

- » 1.Dades d'usuari (user_id, nom_complet, email).
- » 2.La data i l'import exacte de la 3a compra.
- » 3.Mitjana de les 3 primeres: La mitjana de despesa de les seves transaccions 1, 2 i 3.

Pista Tècnica:

- » Has d'utilitzar la funció ROW_NUMBER() per establir l'ordre cronològic de les compres.
- » Has d'utilitzar la clàusula QUALIFY per filtrar.
- » Repte d'Optimització: Busca l'estratègia de menor cost computacional. Tingues en compte que calcular mitjanes sobre tot l'historial històric és costós. Intenta filtrar les files necessàries abans de realitzar l'agregació final.

Esta query la resolvimos con CTEs. Primero creamos la tabla 'dades', donde incorporamos los datos de los usuarios.

A continuación, hicimos la tabla 'tercera_compra', donde usamos ROW_NUMBER, PARTITION_BY y ORDER BY ASC para ordenar las transacciones en orden ascendente de acuerdo al campo 'timestamp_50_days' para cada usuario. Usamos la cláusula QUALIFY como filtro para así tan solo quedarnos con los datos de la tercera compra: fecha e importe.

Después, hicimos una tercera tabla donde volvemos a usar ROW_NUMBER() y PARTITION_BY usuario, ordenando de acuerdo al campo 'timestamp_50_days' de manera ascendente. Hemos vuelto a usar QUALIFY, pero en este caso, hemos seleccionado las tres primeras compras para usar la función AVG(amount) sobre ellas una vez hemos agrupado por usuario.

Finalmente, hemos seleccionado los campos solicitados y hemos unido las diferentes tablas mediante el campo 'userd_id'.

En la imagen 15 mostramos el código explicado anteriormente y las primeras filas del resultado.

```

35 WITH tercera_compra AS (
36     SELECT user_id,
37           ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp_50_days ASC) AS list_compra,
38           timestamp_50_days AS data_tercera_compra,
39           amount AS import_tercera_compra
40     FROM `sprint3-analytics-arnau-rg.sprint3_gold.fact_transactions_optimized`
41    WHERE declined = 0
42   QUALIFY list_compra = 3),
43
44   mitja_trans AS (SELECT user_id, ROUND(AVG(amount),2) AS mitjana_tres_primeres_trans
45 FROM
46   (SELECT user_id,
47         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY timestamp_50_days ASC) lista,
48         amount
49    FROM `sprint3-analytics-arnau-rg.sprint3_gold.fact_transactions_optimized`
50   WHERE declined = 0
51   QUALIFY lista <= 3) AS d
52 GROUP BY user_id)
53
54 SELECT d.user_id,
55        CONCAT(d.name, ' ', d.surname) AS nom_complet,
56        d.email,
57        tc.data_tercera_compra,
58        tc.import_tercera_compra,
59        mt.mitjana_tres_primeres_trans
60 FROM tercera_compra AS tc
61 JOIN `sprint3_silver.users_combined` AS d
62 ON tc.user_id = d.user_id
63 JOIN mitja_trans AS mt
64 ON d.user_id = mt.user_id;

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta [Chat](#) [Guardar los resultados](#) [Abrir en](#)

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución	
Fila	user_id	nom_complet	email	data_tercera_compra	import_tercera_compra	mitjana_tres_pri...
1	203	Jarrold Fields	sit.amet@google.couk	2026-03-15 11:41:38.353144 UTC	484.41	378.29
2	1772	Rysxpk Jkfeheuh	rysxpk.jkfeheuh@example.com	2026-03-19 11:41:38.353144 UTC	213.36	229.31
3	1497	Etcuyh Irysyzge	etcuyh.iryzyge@example.com	2026-03-11 11:41:38.353144 UTC	192.19	279.03

Resultados por página: 50 1 - 50 de 5000

Imagen 15

3. Nivell 3

Exercici 1: Desanidament i Aplanament de Dades (Unnesting)

Escenari: Actualment, a la taula de transaccions, els productes venuts estan "amagats" dins d'un Array (una llista d'IDs). L'equip de vendes vol analitzar quins productes específics es venen més, però no poden fer-ho si les dades estan comprimides en una llista. Necessiten una Taula Detallada (Flat Table) on cada fila representi un producte venut, encara que això signifiqui que l'ID de la transacció es repeteixi.

🌱 **Tasca:** Crea la taula `dim_transactions_flat` desnormalitzant la informació. Has d'"explotar" l'array de productes i creuar-lo amb el catàleg mestre per obtenir els noms i preus individuals.

⚙️ Passos Tècnics:

- » 1. Utilitza `CROSS JOIN UNNEST(product_ids)` per transformar l'Array en files individuals.
- » 2. Realitza un `JOIN` amb la taula `products` per obtenir el nom (`name`) i preu (`price`) de cada ítem.
- 📌 Nota: Ves amb compte amb els tipus de dades en fer el `JOIN` (Array de `INT64` vs `Product_ID STRING`).
- » 3. No agrupeu el resultat. Volem veure el desglossament línia per línia.

✅ Resultat Esperat (Visualització):

La teva taula final ha de veure's "plana", on una transacció amb 3 productes ocupa 3 files:

transaction_id	timestamp	total_ticket (Global)	product_sku	product_name	product_price
TXN1001	2025-12-08	155.97	SKU_A	Portàtil Bàsic	99.99
TXN1001	2025-12-08	155.97	SKU_B	Ratolí	12.99
TXN1001	2025-12-08	155.97	SKU_C	Funda	29.99
TXN1002	2025-12-07	45.50	SKU_D	Teclat	45.50

👉 Entrega:

- » Una captura de la Vista Prèvia de la teva taula on es vegi el DDL executada correctament.
- » Una captura de pantalla on es vegi l'esquema de la nova taula.

En la imatge 16 podem observar el codi utilitzat per crear la nova taula. Per això, hem fet una join a través de dues taules. Hem recollit els següents dades de la taula `'fact_transactions_optimized'`: `transaction_id`, la data en format `DATE`, el `'amount'` de cada transacció i, usant `CROSS JOIN UNNEST` i `SPLIT`, hem explotat la columna de `'product_ids'` on estaven (en un array).

Els dades dels productes els hem obtingut de la taula `'products_clean'`. Aquests són: `'product_id'`, el nom i el preu del producte.

Finalmente, hemos unido ambas tablas mediante el campo product creado al explotar las filas de los productos de la tabla de transacciones y 'product_id' de la tabla productos. Hemos casteado ambos campos a INT64 para hacer la join. Finalmente, hemos establecido el filtro WHERE para considerar sólo las transacciones aceptadas y hemos ordenado el resultado por transacción para así comprobar los productos asociados para cada transacción.



Imagen 16

En la imagen 17, observamos la vista previa de la tabla creada. Podemos observar, para cada transacción, la fecha en la que se efectuó, el importe total de la transacción y el nombre de los productos que se vendieron con su respectivo precio.

Finalmente, en la imagen 18, observamos el esquema de la nueva tabla, con los diferentes campos creados y el tipo de dato.

sprint3-analytics-arnau-rg / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti... Consulta Chat Abrir en ▾ + Compartir ▾ Copiar

	Esquema	Detalles	Vista previa	Explorador de tablas	Vista previa	Estadísticas	Linaje	
Fila	transaction_id	fecha_transa...	total_ticket	product...	product_name	product_price		
1	00043A49-2949-494B-A5DD-A5...	2026-03-11	395.43	97	jinn Winterfell	65.25		
2	00043A49-2949-494B-A5DD-A5...	2026-03-11	395.43	87	sith Jade	96.26		
3	00043A49-2949-494B-A5DD-A5...	2026-03-11	395.43	26	Stark Karstark	53.01		
4	00043A49-2949-494B-A5DD-A5...	2026-03-11	395.43	16	the duel warden	180.91		
5	000447FE-B650-4DCF-85DE-C7...	2026-03-15	155.63	66	mustafar jinn	2.12		
6	000447FE-B650-4DCF-85DE-C7...	2026-03-15	155.63	69	Littlefinger the giantsblood	57.25		
7	000447FE-B650-4DCF-85DE-C7...	2026-03-15	155.63	87	sith Jade	96.26		
8	00045D6B-ED2E-4F2F-8186-CE...	2026-03-11	326.01	30	Karstark warden	79.53		
9	00045D6B-ED2E-4F2F-8186-CE...	2026-03-11	326.01	81	the duel	15.87		
10	00045D6B-ED2E-4F2F-8186-CE...	2026-03-11	326.01	11	Karstark Dorne	49.7		
11	00045D6B-ED2E-4F2F-8186-CE...	2026-03-11	326.01	16	the duel warden	180.91		
12	000481C3-1C26-4FEF-83A0-4C...	2026-03-31	161.6	72	north	161.6		
13	00051AA4-9CBE-4268-B070-C3...	2026-04-14	148.91	18	Karstark warden	148.91		
14	0008A312-EDFE-4A4F-BC99-E9...	2026-04-05	294.59	19	dooku solo	60.33		
15	0008A312-EDFE-4A4F-BC99-E9...	2026-04-05	294.59	33	duel warden	127.09		
16	0008A312-EDFE-4A4F-BC99-E9...	2026-04-05	294.59	35	the giantsblood maester	107.17		
17	0009A151-9BCF-4E31-9053-A4...	2026-03-14	383.63	91	the duel	59.76		
18	0009A151-9BCF-4E31-9053-A4...	2026-03-14	383.63	55	Tully maester	3.83		
19	0009A151-9BCF-4E31-9053-A4...	2026-03-14	383.63	93	Littlefinger Tarly Stark	192.6		
20	0009A151-9BCF-4E31-9053-A4...	2026-03-14	383.63	28	chewbacca mustafar	127.44		
21	0009D494-6245-4DF9-955D-2C...	2026-04-26	197.8	55	Tully maester	3.83		
22	0009D494-6245-4DF9-955D-2C...	2026-04-26	197.8	8	Winterfell	32.37		
23	0009D494-6245-4DF9-955D-2C...	2026-04-26	197.8	72	north	161.6		
24	000A1DEC-CDB6-4AB2-A619-7...	2026-03-10	339.94	73	Dorne bastard	114.09		

Imagen 17

sprint3-analytics-arnau-rg / Datasets / sprint3_gold / Tables / dim_transactions_flat

☆ dim_transacti... Consulta Chat Abrir en ▾ + Compartir ▾

< Esquema Detalles Vista previa Explorador de tablas Vista previa Est...

✧ Genera descripciones de tablas y columnas con Gemini. Asegúrate de realizar un descripción.

Descartar Generar

Filtro Escribir el nombre o valor de la propiedad

☐	Nombre del campo	Tipo	Modo	Descripción	Clave
☐	transaction_id	STRING	NULLABLE	-	-
☐	fecha_transaccion	DATE	NULLABLE	-	-
☐	total_ticket	FLOAT	NULLABLE	-	-
☐	product_id	INTEGER	NULLABLE	-	-
☐	product_name	STRING	NULLABLE	-	-
☐	product_price	FLOAT	NULLABLE	-	-

Imagen 18

Exercici 2: El Rànquing de Vendes (Agregació Simple)

Escenari: Ara que has preparat la infraestructura i disposes de la taula `dim_transactions_flat` (on cada fila equival a un producte venut), generar informes analítics és trivial. Ja no necessites funcions complexes de desanidament per respondre preguntes bàsiques de negoci.

🛠️ **Tasca:** Genera el Top 5 de productes més venuts en la història de la companyia.

🔧 **Mètode:** Consulta directament la teva nova taula desnormalitzada (`dim_transactions_flat`).

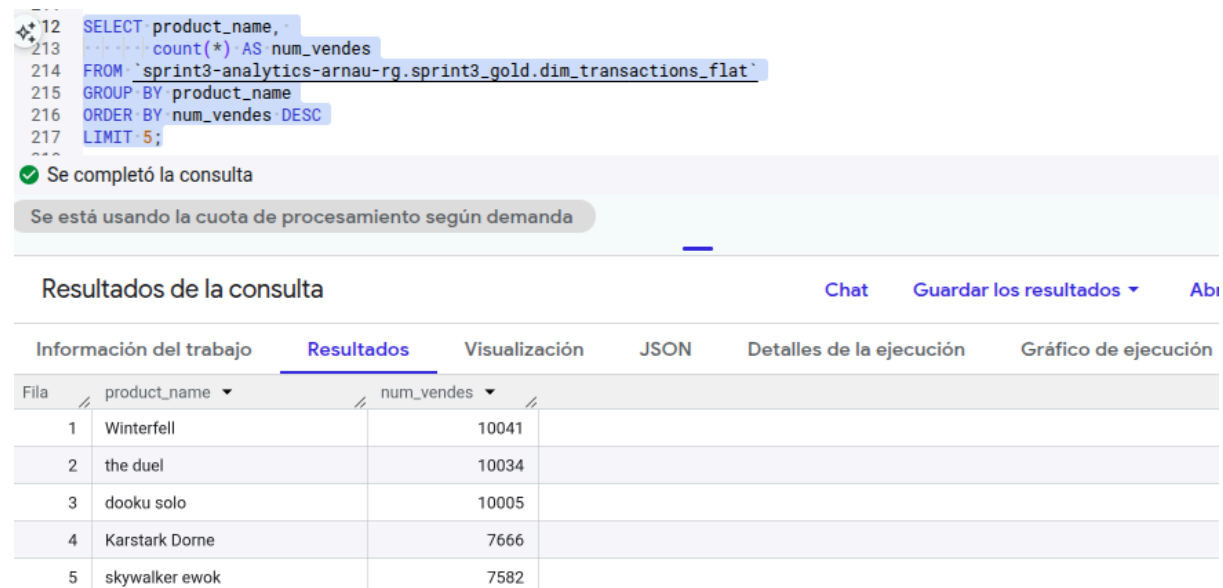
» En ser la taula plana, simplement necessites una agrupació estàndard (GROUP BY) pel nom del producte i un compte (COUNT) de les files.

👉 **Entrega:**

» El codi SQL que genera el rànquing.

» Una captura del resultat amb els 5 productes guanyadors i quantes unitats s'han venut de cadascun.

En la siguiente imagen (imagen 19) mostramos el código utilizado para obtener el nombre de los 5 productos más vendidos con el número de ventas hechas para cada uno de ellos.



```

12 SELECT product_name,
13        count(*) AS num_vendes
14 FROM `sprint3-analytics-arnau-rg.sprint3_gold.dim_transactions_flat`
15 GROUP BY product_name
16 ORDER BY num_vendes DESC
17 LIMIT 5;

```

Se completó la consulta

Se está usando la cuota de procesamiento según demanda

Resultados de la consulta

Chat Guardar los resultados ▾ Abi

Información del trabajo	Resultados	Visualización	JSON	Detalles de la ejecución	Gráfico de ejecución
Fila	product_name	num_vendes			
1	Winterfell	10041			
2	the duel	10034			
3	dooku solo	10005			
4	Karstark Dorne	7666			
5	skywalker ewok	7582			

Imagen 19

Exercici 3: Automatització del Pipeline i Visualització

Escenari: L'informe estàtic de l'exercici anterior ha agradat molt, però el Director de Màrqueting necessita professionalitzar-lo per a la presa de decisions diària. Té dos requeriments urgents:

- » **1. Lògica Centralitzada:** Volen veure el preu amb IVA (21%) desglossat. No obstant això, com que l'impost pot canviar per país o any, et prohibeixen posar el càlcul $\text{preu} * 1,21$ directament en el codi ("Hardcoded"). Has d'utilitzar una funció reutilitzable.
- » **2. Frescor de Dades:** El tauler ha d'estar actualitzat automàticament cada matí a les 07:00 AM, abans de la reunió diària de vendes.

Tasca:

Has d'evolucionar la teva taula `dim_transactions_flat` per incloure el càlcul d'impostos i automatitzar la seva regeneració.

Passos Tècnics:

- » **1. User Defined Functions (UDF):** Crea una funció SQL persistent anomenada `calculate_tax(amount)` que rebi un valor numèric i retorni el resultat aplicant el 21% d'impost.
- » **2. Integració i Orquestració:**
Modifica el codi de creació de la taula (de l'Exercici 1) perquè utilitzi la teva nova funció `calculate_tax` i generi una columna nova: `product_price_tax_inc`.
Configura una Scheduled Query a BigQuery perquè aquesta sentència (CTAS) s'executi automàticament cada dia a les 07:00 AM.
- » **3. Visualització (BI):** Connecta Looker Studio a la teva taula `dim_transactions_flat` i crea un Dashboard: "Monitor de Rendiment de Vendes"

Entrega:

- » El codi SQL de la UDF.
- » El codi SQL actualitzat de la creació de la taula.
- » Una captura de pantalla demostrant que la Scheduled Query està programada.
- » Un enllaç del gràfic a Looker Studio.

Para hacer la función que nos devuelva el 21% del precio del producto, lo equivalente al impuesto, usamos `CREATE OR REPLACE FUNCTION`. Nombramos a la función como `'calculate_tax'` y el parámetro de entrada como `'price'` (la columna de la tabla `'dim_transactions_flat'`) que es de tipo `FLOAT`. La expresión de la función la establecimos multiplicando el precio del producto por 0,21 (ver en imagen 20).

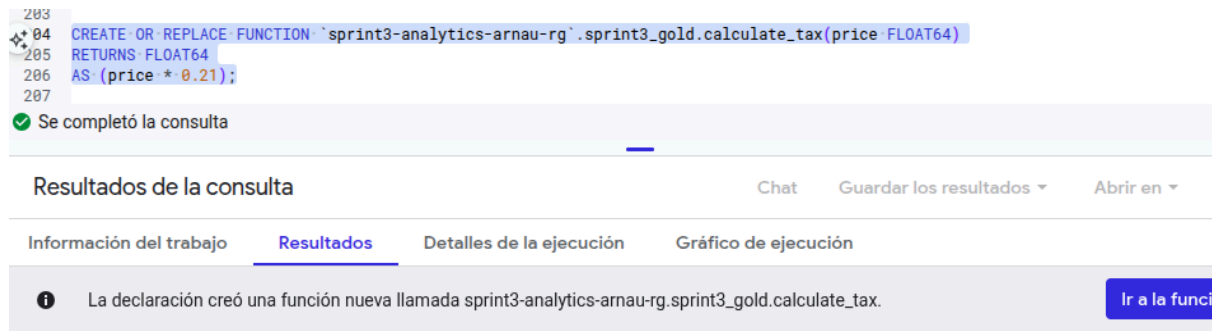


Imagen 20

En la imagen 21, apreciamos el código que habíamos usado para crear la tabla 'dim_transactions_flat', pero en este caso le hemos añadido otro campo llamado 'product_price_tax_inc', que es resultado de aplicar la función anterior sobre el precio de cada producto, el cual nos da el impuesto, y sumarle el precio. Así obtenemos el precio de cada producto con el impuesto incluido.

Podemos observar, más abajo, que en la tabla efectivamente se añadió este nuevo campo. En la imagen 22, mostramos el código utilizado para reemplazar la tabla existente: CREATE O REPLACE TABLE, seguido del código descrito.

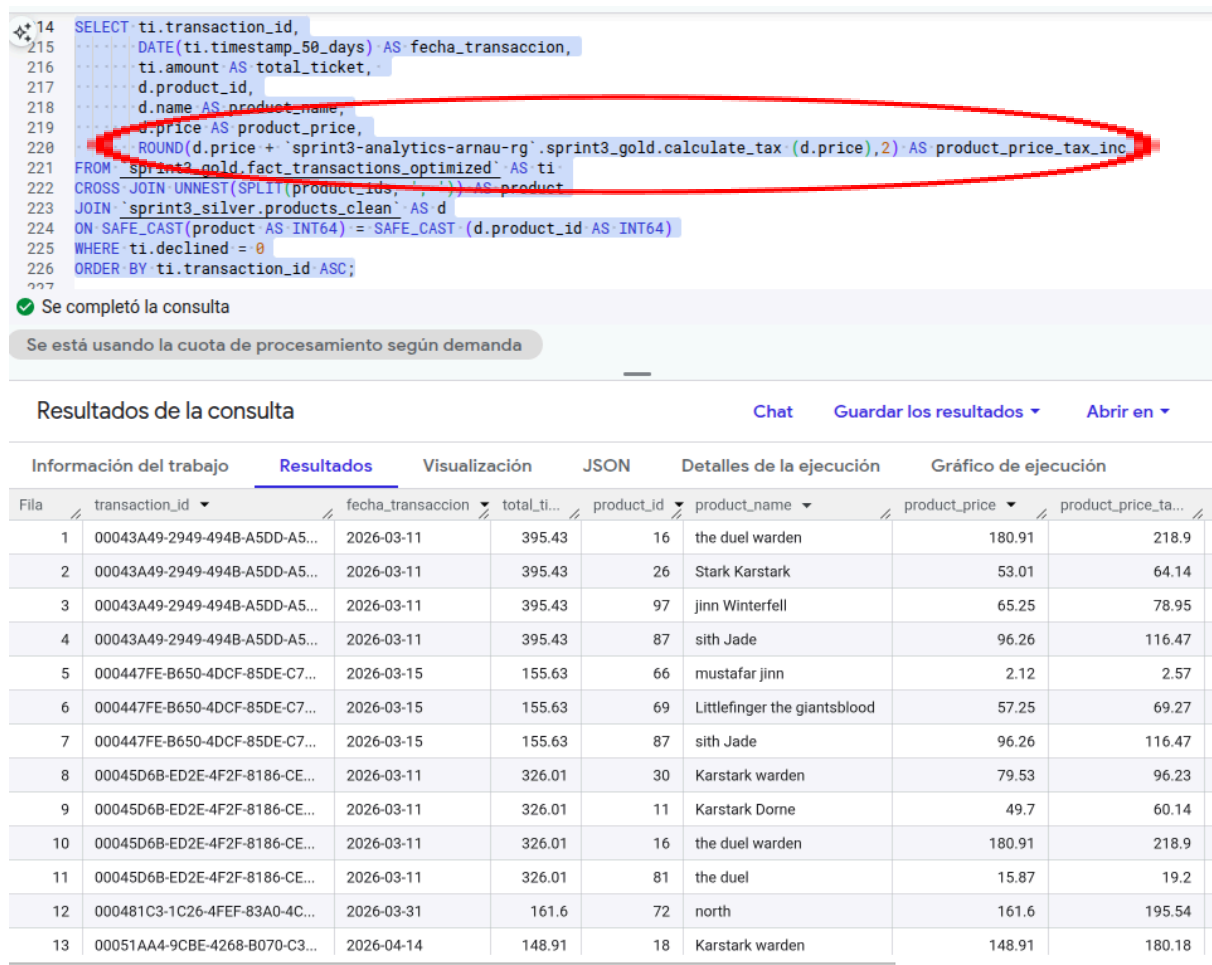


Imagen 21



Imagen 22

En la imagen 23, observamos la consulta que hemos programado para todas las mañanas a la 7:00 AM hora peninsular, que corresponde a las 05:00 UTC. La query programada consiste en el código de creación de la tabla 'dim_transactions_flat', de manera que todas las ventas hechas durante el día aparecerán al crearse la tabla. No obstante, no me dejaba programar esta query, posiblemente debido a algún permiso de mi cuenta que solo se habilita en las cuentas de pago.

Así que probé en programar una query simple (SELECT * FROM 'dim_transactions_flat'). Esta si pude programarla (ver imágenes 24 y 25). Supongo que como esta query no implica la actualización de ningún campo, si me permite programarla.

Finalmente, el informe de ventas se puede ver en el siguiente enlace: [informe ventas](#).

! configuration.query.destinationTable cannot be s...

Detalles y programación

Nombre de la consulta programada *
consulta mañana

Opciones de programación

Frecuencia de repetición *
Días

A la(s) *
5:00 UTC

☒ Comenzar ahora ☐ Comenzar a la hora definida

Fecha de inicio y hora de ejecución
4/5/26, 1:37 p.m.  

☒ Sin hora de finalización programada
☐ Programar una hora de finalización

Fecha de finalización  

i

Programado para ejecutarse todos los días a las 5:00 UTC a partir de ahora.

Destino de los resultados de la consulta

☒ Establecer una tabla de destino para los resultados de la consulta

Conjunto de datos *
☒ sprint3-analytics-arnau-rg.sprint3_gold

Table Id *
dim_transactions_flat

Imagen 23

← Detalles de la consulta programada [Editar](#) [Borrar](#)

Actualización mañana

Historial de ejecuciones Detalles

Nombre del recurso	projects/192228911558/locations/europe/transferConfigs/6a007f56-0000-2d08-9534-089e082f3fd0
Usuario	rgnadata@gmail.com Actualizar credenciales
Fuente	Scheduled Query
Conjunto de datos de destino	sprint3_gold
Tipo de tabla de destino	Native table
Programa (UTC)	every day 05:00
Fecha de inicio (UTC)	No se especificó
Fecha de finalización (UTC)	No se especificó
Tema de Cloud Pub/Sub de notificación	No se especificó
Notificaciones por correo electrónico	No se especificó
Encriptación	Administrada por Google

Detalles de la fuente de datos

Query string	SELECT * FROM `sprint3-analytics-arnau-rg.sprint3_gold.dim_transactions_flat`;
Destination table	dim_transactions_flat
Write preference	WRITE_APPEND
Partitioning field	No se especificó
Destination table KMS key	No se especificó

Imagen 24

Busca flujos de trabajo y programas

Estado	Nombre ↑	Tipo	Fuente	Activadores	Estado (UTC+2)	Acciones
Activo	Actualización mañana	Consulta programada	Query	every day 05:00	🟢🟢🟢	⋮

Imagen 25